Challenges and Resolutions

8Byte.ai DevOps Engineer technical assignment. A record of the non-obvious problems, the reasoning, and the verification.

This document describes the non-obvious problems encountered while building the assignment, the reasoning behind each resolution, and how the result was verified.

1. ECS needs an image before ECR has one

Problem: Terraform creates the ECS service, target group, and task definition, but the application image is built and pushed by CI. A fresh AWS account has an empty ECR repository, so the first ECS task has nothing to pull and the ALB target group never becomes healthy.

Why it matters: The first `terraform apply` should leave a working, observable environment, not a permanently unhealthy service. It also makes the Loom demo dependent on a successful pipeline before the infrastructure can be shown.

Resolution: Terraform uses a tiny public bootstrap image (`public.ecr.aws/docker/library/python:3.12-alpine`) with a one-line HTTP server that returns `200` for all paths. The ECS task definition adds a `command` override only in bootstrap mode. The deployment script regenerates the task definition from the latest family, sets the real image, removes the `command` override, and registers a new revision. The first CI deploy replaces the placeholder without a Terraform apply.

Verification: `terraform validate` passes; the render script removes `command` with `jq`; the ALB health check path `/health/ready` is satisfied by the placeholder during the initial apply and by the Flask app after deployment.

Trade-off: Terraform ignores task definition changes on the ECS service, so CI owns the live revision. The next deployment reads the latest task definition family and picks up Terraform environment/secret changes before applying the new image.

2. Bootstrapping the Terraform backend

Problem: The environment roots need an S3 bucket and DynamoDB lock table, but creating those with the same remote backend is circular. A local-state bootstrap is unavoidable for the first layer.

Resolution: `infra/bootstrap` intentionally uses local state and creates the state bucket, lock table, and GitHub OIDC provider. Each environment then uses a partial S3 backend configured with `-backend-config=../../backend/<env>.hcl`. The bootstrap state is small and documented as something to back up separately.

Verification: `terraform init -backend=false` and `terraform validate` pass for bootstrap and both environments. The backend files are gitignored; only `.example` files are committed.

Trade-off: The bootstrap state contains the state bucket itself, so losing it means importing three resources. A future improvement is a dedicated platform account/stack managed outside the application repository.

3. ACM validation options are a set, not a list

Problem: The first Terraform validation failed while indexing `aws_acm_certificate.this[0].domain_validation_options[0]`. The attribute is a set, so numeric indexes

are not valid.

Resolution: Convert it once with `tolist(...)` and use element zero. The stack creates a single-domain certificate (no SANs), so one validation record is sufficient. `create_acm_certificate` has a lifecycle precondition requiring both `domain_name` and `hosted_zone_id`.

Verification: `terraform validate` passes for both roots. The default tfvars disable ACM so the stack can be evaluated without owning a domain; production documentation calls out enabling TLS before real traffic.

4. ECS service has no `arn` attribute

Problem: The GitHub deployment role policy initially used `aws_ecs_service.app.arn`. Terraform validation failed because the ECS service resource exports `id`, not `arn`.

Resolution: Construct the service ARN explicitly: `arn:aws:ecs:<region>:<account>:service/<cluster>/<service>`. The policy is scoped to that one service, and `iam:PassRole` is restricted to the two ECS task roles with the `ecs-tasks.amazonaws.com` condition.

Verification: `terraform validate` passes; the policy no longer uses a non-existent attribute and remains least-privilege per environment.

5. Circular security-group references

Problem: The ALB security group needs egress to the ECS security group, and the ECS security group needs ingress from the ALB security group. Declaring both as inline rules creates a Terraform dependency cycle. The same pattern appears between ECS and RDS.

Resolution: Split the directions. ALB and ECS security groups declare their egress rules inline; cross-security-group ingress uses `aws_vpc_security_group_ingress_rule` with `referenced_security_group_id`. The RDS security group has an empty egress list and a separate ingress rule from ECS. This breaks the resource-level cycle while keeping the rules least-privilege.

Verification: `terraform validate` passes and the graph has no cycle. The resulting rules are ALB -> app port, app -> RDS 5432, app -> HTTPS for AWS APIs, and DNS to the VPC resolver.

Trade-off: The RDS egress list is explicitly empty rather than inheriting the AWS default allow-all rule; that is intentional and documented in the code.

6. GitHub OIDC subject changes with environments

Problem: A job that uses a GitHub Environment gets an OIDC `sub` claim such as `repo:owner/repo:environment:production`, not `repo:owner/repo:ref:refs/heads/main`. A trust policy written only for `main` works for a branch job but fails for an environment-protected production job.

Resolution: Each environment stack creates a role whose trust policy is scoped to `repo:<owner>/<repo>:environment:<environment>`. The staging and production jobs both declare `environment: staging` / `environment: production`, so the subject is predictable. Production additionally requires reviewers in the GitHub Environment settings. No static AWS keys are stored.

Verification: The trust policy is generated from the same `environment` variable used in the stack and the tfvars. The README lists the exact GitHub variables and environment setup.

Trade-off: The pipeline is intentionally coupled to repository variables. A future improvement is to generate the variable table from Terraform outputs in a small setup script.

7. Python version and the psycopg binary wheel

Problem: The runtime image uses Python 3.12, but the local validation machine has Python 3.9. Installing `psycopg[binary]==3.2.3` failed on 3.9 because that binary wheel is not published for the interpreter. The initial code also used PEP 604 unions (`str | None`), which are only evaluated natively on 3.10+.

Resolution: Pin `psycopg[binary]==3.2.6`, which has a 3.9 wheel and a 3.12 wheel, and add `from __future__ import annotations` where unions are used. The container remains on Python 3.12; local tests run on 3.9. The application avoids runtime-only 3.10 features.

Verification: `pytest` passes on Python 3.9; Ruff targets `py39`; the Dockerfile uses Python 3.12 and installs the same pinned requirements from a wheel cache.

8. Liveness vs readiness during database startup

Problem: If the ALB health check calls a readiness endpoint that queries PostgreSQL while RDS is still creating or the schema is being initialized, the target is marked unhealthy. If the container health check calls the same endpoint, ECS can kill a task that is merely waiting for the database.

Resolution: Separate the probes:

- `/health/live` returns `200` without touching the database. The ECS container

health check and Docker `HEALTHCHECK` use it.

- `/health/ready` runs `SELECT 1` and returns `503` until PostgreSQL is reachable.

The ALB target group uses it.

The entrypoint optionally runs `python -m app.init_db`, which retries schema creation 30 times with a two-second backoff.

Verification: Local Gunicorn + SQLite run returned `200` for both endpoints; unit tests cover both paths. The readiness path is tested in CI against PostgreSQL.

9. CloudWatch EMF inside a JSON logger

Problem: The application wanted both structured JSON logs and custom CloudWatch metrics. If the EMF document is serialized as the `message` field of another JSON object, CloudWatch does not see `_aws` at the root and no metrics are extracted. Adding a CloudWatch agent sidecar would add another container and more IAM.

Resolution: The logging formatter merges a record's `emf` extra into the root of the JSON object. The request middleware builds an EMF document with namespace `8Byte/App`, dimensions `Service`/`Environment`, and `RequestCount`, `ErrorCount`, `ClientErrorCount`, and `Latency` values. The log line remains queryable by `request_id`, `Path`, and `StatusCode`.

Verification: A local Gunicorn run produced a log line with `_aws` at the root and all four metric values. CloudWatch dashboards and alarms reference that namespace and dimensions.

Trade-off: EMF dimensions are deliberately limited to service and environment to avoid high-cardinality metrics. Path and status code remain in logs for debugging but are not dimensions.

10. RDS engine version drift

Problem: RDS returns a resolved minor version (for example `16.4`) even when the configuration asks for a major version (`16`). Terraform can show a perpetual diff, and applying it can trigger unintended maintenance.

Resolution: Keep the configurable major version, enable `auto_minor_version_upgrade`, and add `ignore_changes = [engine_version]` with a comment. Major upgrades are an explicit change, not an incidental plan diff.

Verification: `terraform validate` passes; the parameter group family is derived from the major version (`postgres16`).

11. Availability vs cost in networking and compute

Problem: A textbook HA design uses one NAT gateway per AZ, Multi-AZ RDS, on-demand Fargate, and multi-AZ ALB. That is correct for production but unnecessarily expensive for a short-lived assignment/demo environment.

Resolution: The same module exposes the trade-off through variables:

- Staging: one NAT gateway, single-AZ `db.t4g.micro`, Fargate Spot, one task.
- Production: one NAT per AZ, Multi-AZ `db.t4g.small`, on-demand Fargate, two

tasks, deletion protection, longer backups and logs.

The README makes the trade-off explicit rather than pretending the staging stack is highly available.

Verification: Both tfvars files pass `terraform validate`; the module has validation for AZ count, environment, and monitoring interval.

12. Build once vs per-environment ECR repositories

Problem: Each environment has its own ECR repository, so a staging image is not automatically available to production. Building again after approval risks a different image if the base image or build cache changes.

Resolution: The pipeline deploys the same commit to staging, then after manual approval rebuilds that exact commit and pushes it to the production ECR repository with the same immutable `sha-<commit>` tag. This is simpler than cross-repository promotion and keeps production credentials out of the staging build. The limitation is documented; the roadmap calls for a shared artifact repository or an explicit `docker pull`/retag promotion step with cosign verification.

Verification: The workflow uses `github.sha` for both tags and never deploys a mutable `latest` tag.

13. Docker context and test files

Problem: A naive `COPY . .` in the Dockerfile includes tests, caches, and local environment files, bloating the image and increasing the attack surface.

Resolution: The build context is `./app`, the Dockerfile copies only the application package, entrypoint, Gunicorn config, and requirements, and `.dockerignore` excludes tests, caches, dev requirements, and local env files. The builder stage creates wheels; the runtime stage installs from the wheel cache and runs as UID 10001.

Verification: The Dockerfile is reviewed and the pipeline builds it. Docker was not available on the local machine, so the image build itself is verified by CI.

14. Monitoring two different audiences

Problem: One dashboard with every metric is hard to use during an incident. Infrastructure operators need saturation and health; application owners need request rate, error rate, and latency.

Resolution: Two dashboards:

- Infrastructure: ALB, ECS, RDS, storage, connections, host counts.
- Application: EMF request/error/latency, percentiles, error-rate metric math,

top paths, error logs, and RDS correlation.

Alarms are separate from dashboards and route to SNS. Logs Insights widgets are included so an operator can move from a spike to the actual request logs without leaving CloudWatch.

Verification: Dashboard JSON is generated by Terraform `jsonencode`; alarm definitions and dimensions were checked against the metric namespace and dimension names used by the application and AWS services.

15. Bootstrap image lacks curl for the ECS health check

Problem: The ECS task definition used `curl` for the container health check. The first-apply bootstrap image is `python:3.12-alpine`, which does not include `curl`. The ALB health check would pass, but the ECS container health check would fail repeatedly and ECS would replace otherwise healthy tasks.

Resolution: Use a Python standard-library HTTP request for the container health check. Python is present in both the bootstrap image and the application image, so the same task definition works before and after the first CI deploy. The ALB readiness check remains `/health/ready`; the container check remains `/health/live`.

Verification: The bootstrap command and health-check command were executed locally against a temporary HTTP server on port 8011; the health check exited 0.

Trade-off: The health-check command is longer than a `curl` invocation, but it removes a hidden dependency on a tool that is not present in every image.

What I would do differently with more time

- Add an Alembic migration job and run it before the ECS rolling update.
- Add a Terraform plan job on PR using a read-only OIDC role and protected state.
- Promote one signed image between environments instead of rebuilding.
- Add integration tests that hit the ALB in a ephemeral AWS environment.
- Add AWS WAF, GuardDuty, Security Hub, and a customer-managed KMS key.
- Add synthetic canaries and SLO burn-rate alarms.
- Run `checkov`/`tflint` in CI with a reviewed baseline rather than only Trivy.
- Add a restore drill for RDS PITR and document the measured RTO/RPO.